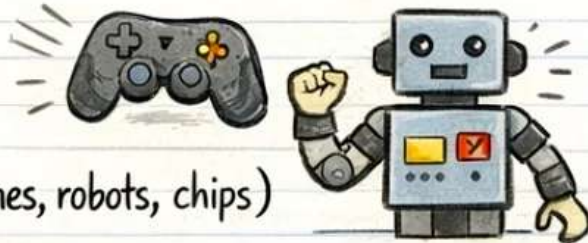


What is C Programming?

C is a computer programming language used to tell the computer what to do.

- It is fast
- It is powerful
- It is used to make:

- ★ Operating systems (like parts of Windows, Linux)
- ★ Games
- ★ Software
- ★ Embedded systems (machines, robots, chips)



👉 C is called a mother language because many languages like C++, Java, Python are inspired by it. @bcawalha

② First C Program: Hello World

```
#include <stdio.h>
int main() {
    printf("Hello World");
    return 0;
}
```

🔍 Explanation (Line by Line)

Code	Meaning
#include <stdio.h>	Adds input-output functions (like printf)
int main()	Program execution starts here
{ }	Inside this, we write program code
printf()	Prints text on screen
return 0;	Ends the program

Basic Structure of a C Program

1 #include <stdio.h>

- This line tells the compiler to include a header file named `stdio.h`.
- `stdio.h` stands for Standard Input Output.
- It allows us to use functions like:
 - `printf()` → for output
 - `scanf()` → for input

 Without this line, the compiler will not understand `printf()` or `scanf()`.

2 int main()

- ✓ ☒ `main()` is mandatory
- `main()` is the starting point of every C program.
- When you run a C program, execution always starts from `main()`, not from the top of the file.

If `main()` is missing: -

- The program will not run
- Think of `main()` as the entry gate of a program.

@bcawalha

3 Why int before main()?

- ✓ `int` means `main()` returns an integer value.
- `return 0;` → program run successfully.
- Any other number → program ended with an error.

4 Program Runs Top to Bottom

- ✓ C programs execute line by line.
- The compiler follows the order from top to bottom, inside `main()`.

5 Semicolon ; is Compulsory

- ✓ Why semicolon is required?

- Semicolon (;) tells the compiler:

"This instruction is complete." right example: -

✓ `printf("Hello");`

✗ `printf("Hello")`

wrong example:

6 return 0;

- Ends the `main()` function.

 Always write `return 0;` at the end `main()`.

Variables in C

- 👉 A variable is a container used to store data in a program.
The value stored in a variable can change during program execution.

example:- `int number = 10;`

Let's break it into parts 🗣️



Parts of a Variable Declaration

1 `int` → Data Type

- Tells the compiler what type of data the variable will store.
- `int` means the variable will store whole numbers.

2 `number` → Variable Name

- It is the name of the container. @bcawalha
- Used to access the stored value in the program.

- 👉 ✓ Must start with a letter or `_`
✓ Cannot start with a number
✓ No spaces allowed.
✓ Cannot use keywords (`int`, `return`, etc.)

📌 Rules for variable names

- ✓ Must start with a letter or `_`
- ✓ Cannot start a number
- ✓ No spaces allowed

✅ Valid:

`int age;            // int_total;`

❌ Invalid:

`int 1num;            // int total marks;`

3 `10` → Value

- This is the data stored in the variable.
- Can be changed later.

✍️ `number = 20;    // value changed`

🔄 Why Variables Are Important?

- ✓ Store data temporarily.
- ✓ Make programs flexible.
- ✓ Help reuse values.
- ✓ Make programs easy to understand.

Data Types in C

👉 Data types tell the computer what type of data you are storing in a variable. Different data types need different amounts of memory and are used for different purposes.

1 int (Integer)

- Used to store whole numbers.
- Can be positive or negative. like:-
`int age = 18; int marks = -5;`

✓ Cannot store decimal values. like:- `int age = 18.6;` ❌

📌 Used for counting, scores, age, roll numbers, etc.

2 float (Decimal Number)

- Used to store decimal (fractional) values.
like:- `float price = 99.50;`
`float pi = 3.14;`

✓ Stores up to 6 decimal digits.
• Takes less memory than double.

📌 Used for prices, averages, temperature, etc.

@bcawalha

3 char (Character)

- Used to store a single character.
like: `char grade = 'A'; char symbol = '@';`

✓ Always written inside single quotes ' '.
• Takes 1 byte of memory.

📌 Used for letters, symbols, and characters.

4 double (Large Decimal Number)

- Used to store large or more precise decimal numbers.
like: `double distance = 12345.678901;` ✓

✓ Stores up to 15 decimal digits.
• More accurate than float.

Input and Output in C

- 👉 Input means taking data from the user.
- 👉 Output means showing data to the user.

In C, input and output are done using built-in functions from `.stdio.h`,

Output using printf()

```
printf("Welcome to C");
```

@bcawalha

✅ Explanation:

- `printf()` is used to display output on the screen.
- Text written inside "" is printed as it is.

👉 Output : Welcome to C

Printing a Variable:

```
int x = 10;  
printf("%d", x);
```

❌ %d is a format specifier for integer values.

Input using scanf()

```
int x;  
scanf("%d", &x);
```

✅ Explanation:

- `scanf()` is used to take input from the user.
- %d tells the compiler that input is an integer.
- &x stores the input value inside variable x.

👉 Important Points.

- %d = integer.
- %d is a format specifier, Used for int data type.
- & = Address of Variable.

13. BREAK, CONTINUE AND GOTO STATEMENTS IN C

These statements are used to change the normal flow of control in loops and programs.

- ★ **break** : Used to terminate the loop or switch case.
- ★ **continue** : Used to skip the current iteration and continue with next iteration.
- ★ **goto** : Used to transfer control to a labeled statement.

Why Break and Continue?

- They help in decision making inside loops.
- They improve the efficiency of code.
- goto is rarely used but useful in certain situations.

1. break Statement

Terminates the loop or switch case immediately.

Syntax :

```
break;
```

Example Program :

```
#include <stdio.h>
int main()
{
    int i;
    for (i = 1; i <= 10; i++)
    {
        if (i == 6)
            break; // loop will terminate
        printf("%d ", i);
    }
    return 0;
}
```

Output :

```
1 2 3 4 5
```

Explanation :

When i becomes 6, break is executed and loop terminates. Numbers 1 to 5 are printed.

2. continue Statement

Skips the current iteration and continues with the next iteration.

Syntax :

```
continue;
```

Example Program :

```
#include <stdio.h>
int main()
{
    int i;
    for (i = 1; i <= 5; i++)
    {
        if (i == 3)
            continue; // skip when i = 3
        printf("%d ", i);
    }
    return 0;
}
```

Output :

```
1 2 4 5
```

Explanation :

When $i = 3$, continue is executed. So 3 is skipped and loop continues with next iteration.

3. goto Statement

Transfers the control of program to another labeled statement.

Syntax :

```
goto label;
```

```
label :
    // statements;
```

Example Program :

```
#include <stdio.h>
int main()
{
    int i = 1;
start: // label
    if (i <= 5)
    {
        printf("%d ", i);
        i++;
        goto start; // jump to label
    }
    return 0;
}
```

Output :

```
1 2 3 4 5
```

Explanation :

Control jumps to label 'start' again and again until i becomes 6. Then condition fails and program exits.

Note :

- ★ goto makes code hard to read.
- ★ It should be used only when necessary.
- ★ break and continue are preferred over goto.

Comparison Table

Statement	Used For	Where Used	Effect	Example Use
break	Exit from loop or switch case	loops, switch	Terminates the loop or exits from switch	Finding a number, exiting loop early
continue	Skip current iteration	loops	Skips remaining code in current iteration	Ignoring some values in loop
goto	Jump to labeled statement	anywhere in function	Transfers control to label	Rare use, error handling, jumping out of nested loops

14. FUNCTIONS IN C

A function is a block of code that performs a specific task. Functions help in breaking a large problem into smaller parts, making the code modular, reusable and easy to maintain.

Advantages of Functions

- ★ Breaks a large program into smaller modules.
- ★ Code reusability - same function can be called multiple times.
- ★ Improves readability and reduces code redundancy.
- ★ Easy to test, debug and maintain.

Why Functions ?

- They reduce the size of the program.
- Increase code reusability.
- Make the program easy to understand.
- Support modular programming.

Types of Functions

1. Library (Built-in) Functions
2. User Defined Functions
 - (a) Without Arguments and Without Return Value
 - (b) Without Arguments and With Return Value
 - (c) With Arguments and Without Return Value
 - (d) With Arguments and With Return Value

1. Library (Built-in) Functions

These functions are predefined in C library (header files).

Example : printf(), scanf(), strlen(), sqrt(), pow() etc.

Example Program : Using sqrt()

```
#include <stdio.h>
#include <math.h>
int main()
{
    double x, s;
    x = 25.0;
    s = sqrt(x);
    printf("Square root of %.2lf = %.2lf", x, s);
    return 0;
}
```

Output :

Square root of
25.00 = 5.00

2. User Defined Functions

Functions created by the user to perform specific tasks.

(a) Without Arguments and Without Return Value

Example Program :

```
#include <stdio.h>
void greet(); // function declaration
int main()
{
    greet(); // function call
    return 0;
}
void greet() // function definition
{
    printf("Hello, Welcome to C Programming!\n");
}
```

Output :

Hello, Welcome
to C Programming!

(b) Without Arguments and With Return Value

Example Program :

```
#include <stdio.h>
int getNumber(); // function declaration
int main()
{
    int n = getNumber(); // function call
    printf("Number = %d\n", n);
    return 0;
}
int getNumber() // function definition
{
    int x = 10;
    return x;
}
```

Output :

Number = 10

(c) With Arguments and Without Return Value

Example Program :

```
#include <stdio.h>
void printSum(int a, int b); // declaration
int main()
{
    printSum(5, 7); // function call
    return 0;
}
void printSum(int a, int b) // definition
{
    int sum = a + b;
    printf("Sum = %d\n", sum);
}
```

Output :

Sum = 12

(d) With Arguments and With Return Value

Example Program :

```
#include <stdio.h>
int add(int a, int b); // declaration
int main()
{
    int s = add(8, 12); // function call
    printf("Sum = %d\n", s);
    return 0;
}
int add(int a, int b) // definition
{
    return a + b;
}
```

Output :

Sum = 20

Function Structure

```
return_type function_name(parameter_list) ← Function Header
{
    // local variable declaration
    // statements
    return value; // if return type is not void ← Return Statement
}
```

← Function Body

← Return Statement

Notes :

- ★ In C, every program must have at least one function - main().
- ★ A function can call another function.
- ★ The return statement is optional in void functions.
- ★ Function declaration (prototype) should be written before main() or before its first use.

15. ARRAYS IN C

An array is a collection of elements of the same data type stored in contiguous (continuous) memory locations under a single name.

Why Arrays ?

- To store multiple values of same type.
- Easy to access using index.
- Reduces code and improves readability.
- Important for handling large data.

1. Declaration of Array

```
data_type array_name[size];
or
data_type array_name[size]={list of values};
```

Examples :

- * `int marks[5];` // declaration of integer array
- * `float salary[10];` // declaration of float array
- * `char name[6] = {'S','h','u','b','h','a','m'};`

2. Initialization of Array

(a) Partial Initialization

```
int arr[5] = {10, 20, 30};
// remaining elements will be 0

Array index : 0 1 2 3 4
Value       : 10 20 30 0 0
```

(b) Complete Initialization

```
int arr[5] = {10, 20, 30, 40, 50};

Array index : 0 1 2 3 4
Value       : 10 20 30 40 50
```

Note :

- ★ Array index always starts from 0.
- ★ Size must be a positive integer (constant).
- ★ Elements are stored in contiguous memory locations.

3. Accessing Array Elements

```
array_name[index];
```

Here, index must be between 0 to size-1.

Example :

```
int arr[5] = {5, 10, 15, 20, 25};
printf("%d", arr[0]); // Output : 5
printf("%d", arr[4]); // Output : 25
```

Memory Representation

```
int arr[5] = {5, 10, 15, 20, 25};
Index : 0 1 2 3 4
Value : 5 10 15 20 25
Address: 1000 1004 1008 1012 1016
(Assume each int takes 4 bytes)
```

4. Working with Arrays (Example Program)

```
#include <stdio.h>
int main()
{
    int i, arr[5];
    // Input
    printf("Enter 5 numbers:\n");
    for(i = 0; i < 5; i++)
        scanf("%d", &arr[i]);

    // Output
    printf("Array elements are:\n");
    for(i = 0; i < 5; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Sample Run :

```
Enter 5 numbers:
1 2 3 4 5
Array elements are:
1 2 3 4 5
```

5. Types of Arrays

Type of Array	Description	Example
One Dimensional	Elements arranged in a single row	<code>int arr[5];</code>
Two Dimensional	Elements arranged in rows and columns	<code>int arr[3][4];</code>
Multi Dimensional	More than two dimensions	<code>int arr[2][3][4];</code>
Character Array	Array of characters (String)	<code>char str[10];</code>
Array of Pointers	Array whose elements are pointers	<code>int *ptr[5];</code>

6. Important Points

- ★ Array size must be defined at compile time.
- ★ Array index should not be out of range, otherwise it causes runtime error.
- ★ Array name represents base address of first element.
- ★ In C, there is no bounds checking for arrays.
- ★ Arrays are passed to functions by reference (address).

Common Error

Accessing invalid index.

Example : `int arr[5];`

`arr[5] = 10;` // Wrong (Index out of range)

Common Interview Questions

Q1. What is an array in C?

Ans: An array is a collection of same type of

Q4. What happens if we access `arr[10]` in array `arr[10]`?

Ans: It causes runtime error because valid index range is 0 to 9.

16. STRINGS IN C

A string in C is an array of characters that is terminated by the null character '\0'.

Key Points

- ★ Strings are stored as character arrays.
- ★ The last character must always be '\0'.
- ★ C does not have a built-in string type.
- ★ String handling is done using `<string.h>` library functions.

Why Strings ?

- Used to store text.
- Important in real world programs.
- Many library functions are available to manipulate strings easily.

1. Declaration and Initialization

➤ Using Character Array

```
char str[20];
str[0] = 'H';
str[1] = 'i';
str[2] = '!';
str[3] = '\0'; // null character
```

➤ Using String Literal

```
char str[1] = "Hi!";
// compiler automatically
// adds '\0' at the end
```

2. Input and Output of Strings

Using scanf()

```
char name[20];
printf("Enter name: ");
scanf("%s", name);
printf("Name: %s", name);
```

Using gets() (Not recommended)

```
char name[20];
printf("Enter name: ");
gets(name);
printf("Name: %s", name);
```

Note : gets() is unsafe, use fgets() instead.

3. Common String Functions (from <string.h>)

Function	Description	Example	Example Use
strlen(str)	Returns length of string.	strlen("Hello")	int len = strlen(str);
strcpy(dest, src)	Copies src string to dest.	strcpy(a, b)	strcpy(dest, src);
strcat(dest, src)	Concatenates src to dest.	strcat(a, b)	strcat(dest, src);
strcmp(str1, str2)	Compares two strings. Returns 0 if equal.	strcmp("hi", "hi")	if (strcmp(a, b) == 0)
strrev(str)	Reverses the string.	strrev("abc")	strrev(str);
strlwr(str)	Converts to lowercase.	strlwr("ABC")	strlwr(str);
strupr(str)	Converts to uppercase.	strupr("abc")	strupr(str);

4. Example Program

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[20], str2[20];
    printf("Enter first string: ");
    scanf("%s", str1);
    printf("Enter second string: ");
    scanf("%s", str2);
    printf("Length of str1 = %d\n", strlen(str1));
    strcpy(str2, str1); // copy
    strcat(str2, " World"); // concatenate
    printf("After strcat: %s\n", str2);
    if (strcmp(str1, "Hello") == 0)
        printf("str1 is equal to Hello\n");
    else
        printf("str1 is not equal to Hello\n");
    return 0;
}
```

5. Memory Representation of String "Hi"

Index	0	1	2
Char	'H'	'i'	'\0'
ASCII	72	105	0

Here, '\0' (null) marks the end of the string.

6. Important Points

- ★ String length does not include the null character.
- ★ Array index starts from 0.
- ★ Always leave enough space for the null character.
- ★ %s in scanf() stops reading at first space.
- ★ Use fgets() instead of gets() to read entire line.

Common Error

If null character is missing, output will continue until a '\0' is found in memory.

Example :

```
char str[5] = {'H', 'i'}; // Missing '\0'
Output may be : Hi??????
```